

The Thiele Machine

A Complete Mathematical Specification

Derived from the Fully Machine-Checked Coq Corpus

Devon Thiele

April 2026

Abstract

This is the complete mathematical specification of the Thiele Machine: a formal model of computation where structural information carries an explicit, conserved cost μ . Everything here is built from the machine-checked Coq corpus. You can reconstruct the machine from what's in this document alone.

Coq establishes that conclusions follow from axioms. It does *not* establish that axioms model physical reality. That distinction matters and I keep it explicit throughout. Every result is tagged as one of three types: **(S)** structural theorems about the machine as a mathematical object, **(C)** conditional derivations where a physics conclusion follows from a stated axiom, and **(R)** consistency relations that check internal coherence but are not independent predictions.

The document covers: the full state space and operational semantics; No Free Insight; μ -ledger necessity and minimality; computational universality and Turing strictness; gauge symmetry and Noether conservation; conditional connections to quantum mechanics (Born rule, Tsirelson bound, no-cloning, unitarity, complex amplitudes, Schrödinger equation) under named physical axioms; spacetime emergence and the Landauer bridge; the Thiele Manifold and self-reference tower; and falsifiable predictions with concrete failure modes.

Contents

1	Introduction	4
1.1	Epistemological Framework	4
2	The State Space	6
2.1	Primitives	6
2.2	Regions and Modules	6
2.3	The Partition Graph	6
2.4	Machine State	6
3	Instruction Set and Cost Model	7
3.1	Instruction Categories	7
3.1.1	Structural Operations	7
3.1.2	Logical Operations	7
3.1.3	Computational Operations (Reversible ALU)	7
3.1.4	Standard ALU and Control Flow	8
3.1.5	Auxiliary Operations	8
3.1.6	Categorical Morphism Operations	8
3.2	Cost Model	9
4	Operational Semantics	10
4.1	Transition Function	10
4.2	Traces and Execution	10
4.3	Selected Instruction Semantics	10
5	No Free Insight	11
5.1	Definitions	11
5.2	The Theorem	11
5.3	μ -Ledger Necessity	11
5.4	Partition Refinement and No Free Insight	13
6	The μ-Chaitin Theorem	14
7	Cost Equals Kolmogorov Complexity	15
8	No Free Lunch and No Arbitrage	16
8.1	No Free Lunch (Ghosts Are Impossible)	16
8.2	No Arbitrage Implies Potential Function	16
9	Computational Universality and Subsumption	17
9.1	Turing Machine Embedding	17
9.2	Strict Containment	17
9.3	Halting Undecidability	17

9.4	Oracle Cost Lower Bound	17
9.5	Confluence	18
9.6	Tensoriality	18
9.7	Amortized Analysis	18
9.8	μ -Complexity Classes	18
10	μ-Conservation and Receipt Integrity	20
10.1	The Conservation Law	20
10.2	Receipt Integrity	20
11	Gauge Symmetry and Noether's Theorem	21
11.1	The $\mathbb{Z}$ -Action (Gauge Group)	21
11.2	Gauge Invariance	21
12	Two-Dimensional Amplitude Space	22
13	Complex Amplitudes from Norm Preservation	23
14	The Born Rule	24
15	Unitarity	26
16	Purification	27
17	No-Cloning	28
18	Observation and Collapse	29
18.1	Observation Irreversibility	29
18.2	Collapse Determination	29
19	The Tsirelson Bound	30
20	Witness Insight Taxonomy	31
21	Bell's Theorem in Morphism Coupling Language	32
22	The Schrödinger Equation	33
23	Planck's Constant	34
24	Emergent Spacetime	35
24.1	The μ -Metric	35
24.2	Causal Cones	35
24.3	No-Signaling	35
24.4	Four-Dimensional Spacetime Geometry	35
24.4.1	4D Simplicial Complex	36
24.4.2	Metric Tensor from μ -Costs	36
24.4.3	Discrete Differential Geometry	36
24.4.4	Curvature Tensors	37
24.4.5	Stress-Energy Tensor	37
24.4.6	Einstein's Field Equations	38
24.4.7	Empirical Validation	40
24.4.8	Affine Metric-Scaled Operator: Non-Uniform Boundary Results	40

25 Information Causality	42
26 Thermodynamic Bridge	43
26.1 Landauer’s Principle	43
26.2 Dissipative Embedding	43
27 Self-Reference and the Thiele Manifold	44
27.1 Self-Reference Escalation	44
27.2 The Thiele Manifold	44
27.3 Genesis: Process–Machine Isomorphism	44
28 Three-Layer Isomorphism	45
28.1 RTL Coverage: Zero Gaps	45
29 The Curry–Howard–Thiele Correspondence	46
30 Categorical Structure	47
30.1 The Morphism Layer	47
30.1.1 Objects and Morphisms	47
30.1.2 The Seven Categorical Opcodes	47
30.1.3 Category Laws	48
30.1.4 The Categorical Separation Theorem	48
30.2 The Conservation as Functor (Archived)	48
31 Audit Infrastructure	49
32 Falsifiable Predictions	50
32.1 Linear Scaling of Structural Cost	50
32.2 CHSH Regime Separation	50
32.3 Metric Deformation	50
32.4 Architectural Permanence	50
32.5 Coupling Constant Prediction	50
A Proof File Index	52
B Epistemological Classification	54

Chapter 1

Introduction

The Thiele Machine makes the cost of structure explicit. A Turing Machine treats all state transitions as cost-equivalent. The Thiele Machine assigns a specific cost μ to every operation that extracts or asserts structural information. That’s the design. Here is where it goes.

The central claim, proven formally: *you cannot gain structural insight about a system without paying μ -cost*. Combined with explicit physical axioms (superposition, linearity, information conservation), that constraint produces structural results that parallel quantum mechanics, thermodynamics, and emergent spacetime. Each result’s status — whether it is a structural theorem **(S)**, a conditional derivation **(C)**, or a consistency relation **(R)** — is marked directly. Nothing is hidden.

Eight parts:

1. **Foundations:** State space, instruction set, operational semantics.
2. **Core Theorems:** No Free Insight, μ -ledger necessity, μ -Chaitin, cost = complexity.
3. **Computation:** Universality, strict subsumption, confluence, halting.
4. **Symmetry & Conservation:** μ -conservation, Noether/gauge, receipts.
5. **Quantum Mechanics:** Born rule, Tsirelson, no-cloning, unitarity, complex amplitudes, Schrödinger, Planck.
6. **Emergent Structure:** Spacetime metric, causal cones, information causality, Landauer bridge.
7. **Meta-Theory:** Self-reference, Thiele Manifold, Genesis, three-layer isomorphism, Curry–Howard–Thiele.
8. **Predictions:** Falsifiable divergences from standard QM.

1.1 Epistemological Framework

Formal verification (Coq) establishes that conclusions follow logically from axioms. It does **not** establish that axioms model physical reality. This distinction is critical for evaluating the physics claims in Parts V–VIII.

Every result in this specification falls into one of three categories:

- (S) Structural** Theorems about the Thiele Machine as a mathematical object. These are unconditionally true given the definitions. *Examples:* μ -conservation, gauge invariance, confluence, halting undecidability, strict Turing subsumption.

- (C) Conditional** Theorems of the form “if axiom X holds, then physics result Y follows.” The logical inference is verified; whether axiom X holds in nature is an empirical question. *Examples:* Born rule uniqueness (conditional on linearity), complex necessity (conditional on 2D amplitudes), unitarity (conditional on information conservation).
- (R) Consistency** Algebraic identities or definitional equivalences that verify internal coherence but do not constitute independent predictions. *Examples:* Planck consistency relation, Tsirelson cost definition.

Each major theorem in Parts V–VIII is annotated with its category. A theorem marked **(C)** or **(R)** is not diminished. Conditional derivations and consistency checks are valuable scientific tools. But the reader should not mistake them for derivations from first principles without stated assumptions. See Appendix B for the complete classification table.

Chapter 2

The State Space

2.1 Primitives

Let $\mathbb{N} = \{0, 1, 2, \dots\}$. Let $\mathbb{B} = \{0, 1\}$. Let Σ be a finite alphabet.

2.2 Regions and Modules

Definition 2.1 (Region). A **Region** R is a finite subset of $\mathbb{N}$, canonically represented as a deduplicated list preserving first-occurrence order: $\text{norm}(L) = \text{nodup}(L)$ removes duplicate elements while preserving the order of their first appearance. (The Python VM applies a stricter canonical form via `sorted(set(region))`; the Coq specification uses `nodup Nat.eq_dec` which does not sort.)

Definition 2.2 (Module). A **Module** $M = (R_M, A_M)$ consists of a normalized region $R_M \subset \mathbb{N}$ and a set of axioms $A_M \subset \Sigma^*$.

2.3 The Partition Graph

Definition 2.3 (Partition Graph). A partition graph $G = (N_{id}, \mathcal{T})$ where $N_{id} \in \mathbb{N}$ is the next available module ID and $\mathcal{T} : \{0, \dots, N_{id} - 1\} \rightarrow \mathcal{M}$ is a finite partial function from IDs to modules.

Axiom 2.1 (Well-Formedness). G is well-formed iff $\text{dom}(\mathcal{T}) \subseteq \{0, \dots, N_{id} - 1\}$.

2.4 Machine State

Definition 2.4 (VM State). The complete state is a 12-tuple:

$$S = (G, C, R, Mem, PC, \mu, T_\mu, err, \ell, m, w, cert)$$

where $G \in \mathcal{G}$ is the partition graph, $C \in \mathbb{N}^4$ comprises the CSRs (certificate address, status, error code, heap base), $R \in \mathbb{N}^{32}$ is the register file, $Mem \in \mathbb{N}^{65536}$ is main memory, $PC \in \mathbb{N}$ is the program counter, $\mu \in \mathbb{N}$ is the μ -ledger, $T_\mu \in \mathbb{N}^{16}$ is the flattened 4×4 μ -tensor (row-major), $err \in \mathbb{B}$ is the global error flag (latching), $\ell \in \mathbb{N}$ is the logic-engine accumulator, $m \in \mathbb{N}$ is the machine status register ($0 = \text{Turing mode}$, $1 = \text{Thiele mode}$), $w \in \mathbb{N}^8$ is the WitnessCount record (`vm_witness`): eight natural-number counters `wc_same_ab` and `wc_diff_ab` for each setting pair $(a, b) \in \{00, 01, 10, 11\}$, and $cert \in \mathbb{B}$ is the certified flag (`vm_certified`).

Chapter 3

Instruction Set and Cost Model

3.1 Instruction Categories

The instruction set $\mathcal{I}$ is organized into six families. Every instruction i carries an explicit cost parameter $\Delta\mu_i \in \mathbb{N}$.

3.1.1 Structural Operations

These modify the partition graph G :

- **PNEW**($R, \Delta\mu$): Create module with region R .
- **PSPLIT**($m, L, R, \Delta\mu$): Split module m into disjoint regions.
- **PMERGE**($m_1, m_2, \Delta\mu$): Merge two modules.
- **PDISCOVER**($m, E, \Delta\mu$): Discovery accounting for module m . The abstract specification attaches evidence E (list of axioms) to the module; the current hardware-aligned step advances PC and charges $\Delta\mu$ without mutating the graph.
- **MDLACC**($m, \Delta\mu$): Minimum-description-length accumulate on module m .

3.1.2 Logical Operations

These interact with information content:

- **LASSERT**($freg, creg, kind, flen, cost$): Assert a formula read from `vm_mem[R[freg]]` (encoded-unit count $flen$, eight concrete bits per unit) against a witness package at `vm_mem[R[creg]]`. In SAT mode the package contains one satisfying assignment and one falsifying assignment, stored back-to-back; $\Delta\mu = flen \times 8 + S(cost) \geq 1$.
- **LJOIN**($c1reg, c2reg, \Delta\mu$): Join certificates from two memory-resident strings addressed by registers.
- **REVEAL**($m, n, cert, \Delta\mu$): Reveal n bits of structure from m ; cost is $n + S(\Delta\mu)$.
- **EMIT**($m, p, \Delta\mu$): Emit payload p from module m ; cost is $|p|_{\text{bits}} + S(\Delta\mu)$.

3.1.3 Computational Operations (Reversible ALU)

Reversible register/memory operations:

- **XFER**($dst, src, \Delta\mu$): Copy register src to dst .

- $\text{XOR_LOAD}(dst, addr, \Delta\mu)$: Load memory word at $addr$.
- $\text{XOR_ADD}(dst, src, \Delta\mu)$: $dst \leftarrow dst \oplus src$.
- $\text{XOR_SWAP}(a, b, \Delta\mu)$: Swap registers a and b .
- $\text{XOR_RANK}(dst, src, \Delta\mu)$: $dst \leftarrow \text{popcount}(src)$.

3.1.4 Standard ALU and Control Flow

- $\text{LOAD_IMM}(dst, imm, \Delta\mu)$: Load 32-bit immediate into register.
- $\text{LOAD}(dst, addr, \Delta\mu)$: Load from main memory.
- $\text{STORE}(addr, src, \Delta\mu)$: Store to main memory.
- $\text{ADD}(dst, r_1, r_2, \Delta\mu)$: $dst \leftarrow r_1 + r_2 \pmod{2^{64}}$.
- $\text{SUB}(dst, r_1, r_2, \Delta\mu)$: $dst \leftarrow r_1 - r_2 \pmod{2^{64}}$.
- $\text{JUMP}(target, \Delta\mu)$: Unconditional jump.
- $\text{JNEZ}(r, target, \Delta\mu)$: Jump if $R[r] \neq 0$.
- $\text{CALL}(target, \Delta\mu)$: Push return address, jump to target.
- $\text{RET}(\Delta\mu)$: Pop return address, jump back.

3.1.5 Auxiliary Operations

- $\text{CHSH_TRIAL}(x, y, a, b, \Delta\mu)$: Run a CHSH correlation trial; $x, y, a, b \in \{0, 1\}$ required.
- $\text{CHECKPOINT}(label, \Delta\mu)$: Execution checkpoint (charges μ).
- $\text{READ_PORT}(dst, ch, val, bits, \Delta\mu)$: Read from I/O channel (cert-setter).
- $\text{WRITE_PORT}(ch, src, \Delta\mu)$: Write to I/O channel.
- $\text{HEAP_LOAD}(dst, addr, \Delta\mu)$: Load from heap-relative address.
- $\text{HEAP_STORE}(addr, src, \Delta\mu)$: Store to heap-relative address.
- $\text{HALT}(\Delta\mu)$: Halt execution.

3.1.6 Categorical Morphism Operations

These 7 opcodes (0x27–0x2D) implement the categorical layer over partition modules. See Chapter 30 for laws and proofs.

- $\text{MORPH}(dst, src, tgt, idx, \Delta\mu)$: Create morphism $src \rightarrow tgt$; write morphism ID to $\text{regs}[dst]$.
- $\text{COMPOSE}(dst, f, g, \Delta\mu)$: Compose $f; g$ when $f.tgt = g.src$; errors on type mismatch.
- $\text{MORPH_ID}(dst, m, \Delta\mu)$: Create identity morphism id_m (diagonal relation).
- $\text{MORPH_DELETE}(f, \Delta\mu)$: Remove morphism f ; cascades when modules are split/merged.
- $\text{MORPH_ASSERT}(f, prop, cert, \Delta\mu)$: **Cert-setter**. Charges $S(\Delta\mu) \geq 1$ unconditionally. Even failed attempts (morphism does not exist) incur cost 1. This is the strongest form of the NoFI law: you cannot attempt to certify a relational claim for free.

- **MORPH_TENSOR**($dst, f, g, \Delta\mu$): Tensor product $f \otimes g$ (requires union modules to exist).
- **MORPH_GET**($dst, f, sel, \Delta\mu$): Read morphism metadata: selector 0 = src, 1 = tgt, 2 = coupling length, 3 = is_identity.

The active implementation exposes a 46-opcode ISA (39 original + 7 categorical morphism opcodes). This condensed mathematical specification presents the core instruction families relevant to the kernel semantics and cost model. Every instruction carries an explicit $\Delta\mu$ parameter; the function `instruction_cost` extracts it.

3.2 Cost Model

Every instruction i carries an explicit cost parameter $\Delta\mu_i \in \mathbb{N}$. The function `instruction_cost( $i$ )` extracts this parameter. The cost is applied via `apply_cost( $s, i$ ) =  $s.\mu + \text{instruction\_cost}(i)$` . Since $\Delta\mu_i : \mathbb{N}$, the μ -ledger is monotonically non-decreasing by construction.

The mandatory positive-cost class is: **CERTIFY**, **MORPH_ASSERT**, and **LJOIN** (each charges $S(\Delta\mu) \geq 1$); **EMIT** ($|payload|_{\text{bits}} + S(\Delta\mu)$); **REVEAL** and **READ_PORT** ($bits + S(\Delta\mu)$); and **LASSERT** ($flen \times 8 + S(\Delta\mu)$). These cannot be zero-cost even with $\Delta\mu = 0$. All other instructions charge exactly their encoded $\Delta\mu$, which is 0 in typical programs.

Chapter 4

Operational Semantics

4.1 Transition Function

The transition $\delta : \mathcal{S} \times \mathcal{I} \rightarrow \mathcal{S}$ is defined by:

1. $PC' = PC + 1$ for non-control-flow instructions; JUMP, JNEZ, CALL, and RET set PC to the target address; HALT stops execution.
2. $\mu' = \mu + \Delta\mu_i$.
3. $err' = err \vee \text{fail}(S, i)$.

4.2 Traces and Execution

Definition 4.1 (Trace). A trace $\tau = [i_0, \dots, i_k] \in \mathcal{I}^*$ is a finite sequence of instructions.

Definition 4.2 (Execution). $\text{Run}([], S) = S$; $\text{Run}(i :: \tau', S) = \text{Run}(\tau', \delta(S, i))$.

4.3 Selected Instruction Semantics

PNEW: Normalize region; if already present, idempotent; else add $(R_{\text{norm}}, \emptyset)$ to G .

REVEAL: Update CSRs with checksum of certificate $cert$. Cost: $bits + S(\Delta\mu)$.

LASSERT: Read formula from $\text{vm_mem}[\text{R}[\text{formula_addr_reg}]]$ (encoded-unit count formula_len , eight concrete bits per unit) and witness package from $\text{vm_mem}[\text{R}[\text{cert_addr_reg}]]$. Verify using cert_kind (false = LRAT/UNSAT, true = satisfying model plus falsifying countermodel). If valid, the constraint is certified; if invalid, set $err \leftarrow \text{true}$. Note: in the current hardware-aligned semantics, LASSERT validates the constraint but does not directly mutate the partition graph's axiom set — cert-channel activation happens through the subsequent CERTIFY or MORPH_ASSERT. Cost: $\text{formula_len} \times 8 + S(\mu_delta)$.

Chapter 5

No Free Insight

The central theorem: structural insight is never free.

5.1 Definitions

Definition 5.1 (Receipt Predicate). $P : \mathcal{O} \rightarrow \mathbb{B}$ is a computable function on observation histories.

Definition 5.2 (Strength Ordering). $P_1 \leq P_2$ iff $\forall o. P_1(o) = 1 \implies P_2(o) = 1$. $P_1 < P_2$ iff $P_1 \leq P_2$ and $\exists o. P_2(o) = 1 \wedge P_1(o) = 0$.

Definition 5.3 (Certification). Trace τ **certifies** P iff execution succeeds ($err = 0$), the supra-certificate flag is set, and the output satisfies P .

Definition 5.4 (Structure Addition). $\text{HasStructureAddition}(\tau)$ is true iff τ contains a REVEAL, LASSERT, or equivalent operation.

5.2 The Theorem

Theorem 5.1 (No Free Insight (`NoFreeInsight_Theorem.v`)). *Let $P_{\text{strong}} < P_{\text{weak}}$. If a trace τ starting from a clean state s_0 certifies P_{strong} , then:*

$$\text{Certified}(\tau, P_{\text{strong}}) \implies \text{HasStructureAddition}(\tau)$$

Proof Sketch. The proof is parametric: `NoFreeInsight_Theorem.v` proves the theorem for any system satisfying the `NO_FREE_INSIGHT_SYSTEM` module type interface. The interface requires four contract obligations (not Coq `Axiom` declarations—they are proved separately for the kernel instantiation): (1) μ -monotonicity, (2) a certification specification linking **certifies** to execution, (3) a no-free-insight contract stating that strict strengthening requires structure addition, and (4) that the system has a notion of clean start. The theorem file itself contains zero `Axiom`, zero `Admitted`. The contract obligations are discharged by the kernel instantiation. $\square$

Corollary 5.2 (Cost of Insight). *Gaining insight (strengthening the predicate) implies $\Delta\mu > 0$.*

5.3 μ -Ledger Necessity

Theorem 5.3 (μ -Ledger Necessity and Complete Independence Classification (`NecessityOfMuLedger.v`, `kernel/NecessityAbstract.v`) — (S)). *Five independence results are formally proven, zero Admitted:*

1. $\mu \perp (\text{mem}, \text{regs}, \text{pc})$: no strict-classical oracle recovers μ *turing_ram_mu_necessity*

2. $\text{certified} \perp (\text{mem}, \text{regs}, \text{pc})$: no strict-classical oracle recovers certification `turing_ram_cert_necessity`
3. $\text{certified} \perp (\text{mem}, \text{regs}, \text{pc}, \mu)$: knowing μ does not determine certification `cost_model_cert_necessity`
4. $\mu \perp (\text{mem}, \text{regs}, \text{pc}, \text{certified})$: knowing certification does not determine μ `cert_model_mu_necessity`
5. $\text{vm_graph} \perp (\text{mem}, \text{regs}, \text{pc}, \mu, \text{certified})$: the categorical layer is independent of the full μ -ledger shadow `graph_not_recoverable_from_P_full`

Three-component independence (`thiele_state_three_component_independence`): μ , `vm_certified`, and `vm_graph` are the three provably independent non-classical state components. Each is irrecoverable from any combination of the others plus `(mem, regs, pc)`.

Minimality (`P_full_is_minimal_complete_extension`): `(mem, regs, pc,  $\mu$ , certified)` is the minimal complete extension of classical state for μ and certification. Graph structure is a third independent axis.

Universality (`mu_ledger_necessity_universal`): The separation holds from every VM-State, not only from crafted zero-state witnesses. From any reachable state, `CERTIFY 0` and `PNEW [] 0` produce identical strict classical shadows but different μ and certification.

Trace closure (`po1_trace_necessity`): For any common prefix program of any length, the two witness extensions maintain equal strict shadows but strictly different μ . No accumulation of classical computation closes the gap.

Proof Sketch. Each separation uses explicit witnesses where equal projected inputs force equal oracle outputs, while the witness states require unequal outputs—a contradiction. Strict shadow equality: both `CERTIFY 0` and `PNEW [] 0` advance `pc` by 1 and preserve `mem/regs`. μ divergence: `CERTIFY` charges $S(0) = 1$; `PNEW` charges 0. Graph separation uses `categorical_separation` from `PartitionSeparation.v`: two states with identical P_{full} but different `pg_morphisms`. Universality follows because `vm_apply_certify_strict_shadow` and `vm_apply_pnew_strict_shadow` hold for arbitrary source states. $\square$

Theorem 5.4 (Latent μ : Cost is Intrinsic to Computation (`NecessityOfMuLedger.v §7`) — (S)). *The μ -cost accumulated by any computation is a property of the instruction sequence, not of any tracking system. Four results, all zero Admitted:*

1. **shadow_mu_is_computation_intrinsic**: For any program `prog` and any starting state s :

$$(\text{run_instrs } s \text{ prog}).\mu = s.\mu + \text{trace_total_cost}(\text{prog})$$

The increment is determined entirely by the instruction sequence—independent of `vm_graph`, `vm_regs`, `vm_mem`, `vm_certified`, and the starting μ .

2. **shadow_mu_delta_universal**: Two machines running the same program from any two starting states accumulate the same additional μ . The cost is written in the program before execution begins.
3. **shadow_mu_unique_accounting**: Any instruction-consistent accounting system M satisfying $M(s) = 0$ at the starting state assigns exactly `trace_total_cost` to the result. Replace the Thiele Machine's μ -counter with any other honest accounting and you get the same number.
4. **shadow_mu_inevitable**: Any program containing an instruction with positive declared cost accumulates strictly positive μ from any starting state. There is no free computation.

Interpretation. A Turing machine executing a program pays exactly `trace_total_cost` in latent μ on every execution. The cost was always there—forced by the instruction sequence—but the Turing machine had no mechanism to record or expose it. The μ -ledger does not create this cost. It reveals what every computation was already paying.

5.4 Partition Refinement and No Free Insight

Theorem 5.5 (Partition Ops Are Free (kernel/PartitionRefinementNoFI.v) — (S)). *PNEW, PSPLIT, and PMERGE are not cert-setters and can carry $\Delta\mu = 0$:*

- *partition_structural_ops_not_cert_setters: None of the three partition operations touch `csr_cert_addr` or `vm_certified`.*
- *partition_structural_ops_can_be_free: All three can execute with $\Delta\mu = 0$.*
- *partition_structural_trace_cannot_certify: A trace containing only PNEW, PSPLIT, and PMERGE (all at $\Delta\mu = 0$) cannot reach a certified state from an uncertified start.*

Building partition structure costs nothing by itself. Certifying structural claims about that partition is what costs.

Theorem 5.6 (Partition Free But Certification Non-Free (kernel/PartitionRefinementNoFI.v) — (S)). *partition_free_but_certification_nonfree: There exist traces where all partition-structural instructions carry $\Delta\mu = 0$ (so $\mu_{\text{final}} = \mu_{\text{init}} = 0$) yet the trace ends with a partition graph that a subsequent MORPH_ASSERT or CERTIFY can certify — but only at $\Delta\mu \geq 1$. Partition structure can be built at zero μ -cost; certified partition knowledge cannot be acquired for free. Zero Admitted.*

Theorem 5.7 (Partition Refinement Non-Free (kernel/PartitionRefinementNoFI.v) — (S)). *partition_refinement_nonfree: Any trace that starts with no cert evidence and ends with a certified claim about a partition structure paid at least 1 in μ . This is the NoFI law specialized to partition refinement. Zero Admitted.*

Chapter 6

The μ -Chaitin Theorem

A Chaitin-style incompleteness theorem denominated in μ -currency.

Theorem 6.1 (μ -Chaitin (MuChaitinTheory_Theorem.v)). *For any formal theory system T satisfying the μ -Chaitin interface, the number of bits k the theory can certify is bounded by:*

$$k \leq |T| + c$$

where $|T|$ is the description length of the theory and c is a fixed overhead constant. The proof chain: $k \leq \text{payload} \leq \mu\text{-info} \leq \text{budget} \leq |T| + c$.

Chapter 7

Cost Equals Kolmogorov Complexity

Theorem 7.1 (μ -Bits = Prefix-Free Complexity (CostIsComplexity.v — reference-only)). *Let $K(\text{spec})$ denote the prefix-free Kolmogorov complexity of specification spec . Then:*

$$\mu(\text{spec}) = K(\text{spec})$$

Specifically, $\mu(\text{spec}) = |\text{spec}| + 1$ (the terminating bit), which equals the minimum description length over all prefix-free programs producing spec .

Proof. The canonical compiler maps $\text{spec} \mapsto \text{spec}||[\text{true}]$. Any producing program has the form $p = \text{spec}||[\text{true}]$ with $|p| = |\text{spec}| + 1$. The compiler achieves the minimum, and μ counts exactly these bits. $\square$

Chapter 8

No Free Lunch and No Arbitrage

8.1 No Free Lunch (Ghosts Are Impossible)

Theorem 8.1 (No Free Lunch (NoFreeLunch.v — reference-only)). *A “ghost” is defined as two distinct propositions $p \neq q$ represented by the same physical state. Ghosts are impossible:*

$$\forall p \neq q, \nexists s \text{ s.t. } s \text{ faithfully represents both } p \text{ and } q$$

Information cannot exist without physical distinction.

8.2 No Arbitrage Implies Potential Function

Theorem 8.2 (Potential from No-Arbitrage (NoArbitrage.v — reference-only)). *Let $w : \mathcal{I}^* \rightarrow \mathbb{Z}$ be a cost function satisfying:*

1. **Additivity:** $w([]) = 0$ and $w(\tau_1 \cdot \tau_2) = w(\tau_1) + w(\tau_2)$.
2. **No-Arbitrage:** For every closed cycle τ (returning to the starting state), $w(\tau) \geq 0$.

Then there exists a potential function $\phi : \mathcal{S} \rightarrow \mathbb{Z}$ such that:

$$w(\tau) \geq \phi(\text{apply}(\tau, s)) - \phi(s)$$

for every state s and trace τ .

Remark 8.1. This is structurally analogous to the Second Law of Thermodynamics: consistent cost accounting implies a potential function bounding all transitions. The concrete Coq model is a toy (increment/decrement on nat with asymmetric costs). No temperature, entropy, or thermodynamic system appears in the formalization. The connection to thermodynamics is an analogy, not a derivation.

Chapter 9

Computational Universality and Subsumption

9.1 Turing Machine Embedding

Theorem 9.1 (Abstract Simulation (Embedding_TM.v — reference-only, modular_proofs/TM_to_Minsky.v — reference-only)). *For every Turing Machine T , there exists a Minsky counter machine M and a simulation relation R such that M simulates T step-by-step, encoding the tape into two integers:*

$$\text{Left} = \sum_{i=0}^{h-1} t[h-1-i] 2^i, \quad \text{Right} = \sum_{i=0}^{n-h-1} t[h+i] 2^i$$

The Thiele Machine simulates Minsky using its reversible ALU. Hence:

$$\text{TM} \preceq \text{Minsky} \preceq \text{Thiele}$$

9.2 Strict Containment

Theorem 9.2 (Turing $\subsetneq$ Thiele (kernel/Subsumption.v)). *Classical Turing computation is **properly extended** by sighted Thiele computation (both are Turing-complete, but Thiele carries additional partition and μ -accounting structure):*

$$\exists p. \text{sighted}(p) \wedge \neg \text{turing}(p)$$

There exist programs that are Thiele-computable (using partition structure and μ -accounting) but are not classical Turing programs. The extra structure is genuine new content.

Theorem 9.3 (Semantic Strictness (ThieleFoundations.v — reference-only)). *There exist Thiele traces with identical Turing shadows but non-isomorphic Thiele-level behavior. The partition and μ layers carry information not present in the Turing skeleton.*

9.3 Halting Undecidability

Theorem 9.4 (Diagonal Argument (kernel/OracleImpossibility.v — reference-only)). *No total computable function decides the halting problem for the Thiele Machine.*

9.4 Oracle Cost Lower Bound

Theorem 9.5 (Oracle Impossibility (kernel/OracleImpossibility.v — reference-only)). *The formalization defines a sound oracle as one charging $\Delta\mu \geq 1$ per query (`sound_oracle_cost`). Given this definition:*

1. *oracle_halts_costs_mu*: A zero-cost oracle violates the soundness definition ($0 \geq 1 \Rightarrow \perp$).
2. *oracle_cost_linear*: Given *sound_oracle_cost*, n queries cost $\geq n$.

These results verify the internal consistency of the pricing definition. The physical justification for why oracle queries should cost $\mu > 0$ comes from the No Free Insight framework (gaining information about undecidable propositions should cost something), but the Coq proofs verify the narrower claim that the pricing is self-consistent.

9.5 Confluence

Theorem 9.6 (Church–Rosser (Confluence.v — reference-only)). *For any state s and two independent certificates c_1, c_2 (targeting distinct CNF formulas), applying them in either order yields the same μ -cost:*

$$\mu(s_{12}) = \mu(s_{21})$$

Independent structural operations commute.

9.6 Tensoriality

Theorem 9.7 (Module Independence (ThieleUnificationTensor.v — reference-only)). *For distinct modules $m_1 \neq m_2$, recording discoveries commutes:*

$$\text{record}(\text{record}(G, m_1, e_1), m_2, e_2) \equiv_{\text{ext}} \text{record}(\text{record}(G, m_2, e_2), m_1, e_1)$$

Local per-module operations are tensorial.

9.7 Amortized Analysis

Theorem 9.8 (Amortization (AmortizedAnalysis.v — reference-only)). *For a batch of T instances with discovery cost D and operational cost O per instance:*

$$\text{total_cost} = B \cdot D + T \cdot O \geq T \cdot O$$

where B is batch count. As $T \rightarrow \infty$, average cost $\rightarrow O$ (discovery overhead amortizes away).

9.8 μ -Complexity Classes

Definition 9.1 ($\mu\mathbf{P}$ and $\mu\mathbf{NP}$ (kernel/MuComplexity.v)). Let *PolyBound* be a polynomial $p : \mathbb{N} \rightarrow \mathbb{N}$.

- A decision problem f is in $\mu\mathbf{P}$ iff there exists a polynomial p and a trace family such that for every input state of size n , the trace solves f with μ -cost $\leq p(n)$ and runs in polynomial time.
- A decision problem f is in $\mu\mathbf{NP}$ iff there exists a polynomial p and a verifier trace family such that every YES instance of size n has a certificate verifiable with μ -cost $\leq p(n)$ in polynomial time.

Theorem 9.9 (Classical $P \subseteq \mu P$ (kernel/MuComplexity.v) — (S)). *classical_in_muP: Any classically polynomial-time decision problem (using zero-cost instructions) is in μP with μ -cost = 0 for all inputs. Classical computation is a degenerate case of μ -computation with no structural cost. Zero Admitted.*

Remark 9.1. μP and μNP separate the time cost of computation from the structural cost of certification. Whether $\mu\text{P} \neq \mu\text{NP}$ — or whether there exist problems in μNP not in μP — is an open question. The factored-search witness in `StructuralAdvantage.v` gives a concrete example of a μ -cost advantage (paying $\mu = 18$ to certify a decomposition and reduce search from $O(N^2)$ to $O(2N)$), but that is not a class separation.

Theorem 9.10 (μ -Cost Hierarchy (kernel/MuHierarchyTheorem.v) — (S)). *For every $k \geq 1$, the μ -dimension admits a strict hierarchy:*

1. **Achievability:** *there exists a trace from `init_state` with μ -cost exactly k that achieves `vm_certified = true` and `vm_mu` = k .*
2. **Lower bound:** *any trace from `init_state` achieving `vm_mu` $\geq k$ must have spent at least k units of μ -cost.*

Formally (`mu_hierarchy_theorem`): for every $k \geq 1$, the witness trace `[instr_certify(k-1)]` (`cost` = $S(k-1) = k$) certifies `init_state` at level k , while no trace of μ -cost $< k$ can reach `vm_mu` $\geq k$ from `init_state` (since `trace_mu_cost` = `vm_mu_final` - 0 $\geq k$).

Corollary (`mu_hierarchy_no_upper_bound`): *no fixed μ -budget suffices for all levels. The μ -dimension is unbounded and non-collapsible. Zero Admitted.*

Remark 9.2 (μ -Hierarchy as the formal P-vs-NP analogue). Theorem 9.10 is the μ -analogue of the classical time hierarchy theorem. It formalizes the intuition behind $\text{P} \neq \text{NP}$ in the certified-structure setting: *finding* a certificate of structural complexity k requires paying $\mu \geq k$, while *verifying* it (given the trace) costs 0 additional μ . The hierarchy is strict, infinite, and proven from first principles with zero **Admitted** — no oracle, no diagonalization, no new axioms. The proof uses only `vm_apply_mu` (`cost` = μ increase) and `init_state_mu_zero` (μ starts at 0).

Chapter 10

μ -Conservation and Receipt Integrity

10.1 The Conservation Law

Theorem 10.1 (μ -Ledger Conservation (MuLedgerConservation.v)). *For any trace $\tau = [i_0, \dots, i_k]$:*

$$\mu_{\text{final}} = \mu_{\text{init}} + \sum_{j=0}^k \text{cost}(i_j)$$

Every instruction increases μ by exactly its declared cost. No axioms, no admits.

Corollary 10.2 (Monotonicity). *μ never decreases: $\mu(s) \leq \mu(\text{Run}(\tau, s))$ for all τ .*

Corollary 10.3 (Irreversibility Bound). *$\text{irreversible_bits}(\tau) \leq \mu_{\text{final}} - \mu_{\text{init}}$.*

Theorem 10.4 (μ Decomposition (MuLedgerConservation.v)). *$\mu_{\text{total}} = \mu_{\text{blind}} + \mu_{\text{sighted}}$ for any partition of the cost into blind (reversible) and sighted (structural) components.*

10.2 Receipt Integrity

Definition 10.1 (Receipt). A receipt $r = (\text{step}, \text{instr}, \mu_{\text{pre}}, \mu_{\text{post}}, h_{\text{pre}}, h_{\text{post}})$ records one transition.

Theorem 10.5 (Receipt Integrity (ReceiptIntegrity.v)). *A valid receipt or execution attestation proves $\mu_{\text{final}} = \mu_{\text{init}} + \sum \text{costs}$ for the attested run. Any receipt claiming $\Delta\mu \neq \text{cost}(\text{instr})$ fails validation.*

Theorem 10.6 (Non-Forgeability (ReceiptIntegrity.v)). *Forged receipts (claiming incorrect $\Delta\mu$) fail validation. Overflow values ($\mu > 2^{31} - 1$) are rejected at the receipt layer.*

Chapter 11

Gauge Symmetry and Noether's Theorem

11.1 The $\mathbb{Z}$ -Action (Gauge Group)

Definition 11.1 (Gauge Shift). For $\delta \in \mathbb{Z}$, the gauge shift $\sigma_\delta : \mathcal{S} \rightarrow \mathcal{S}$ maps $S \mapsto S[\mu \leftarrow \mu + \delta]$, preserving all other fields.

Theorem 11.1 (Group Action (KernelNoether.v)). *The gauge shifts form a $\mathbb{Z}$ -action on states:*

1. **Identity:** $\sigma_0(S) = S$.
2. **Composition:** $\sigma_a(\sigma_b(S)) = \sigma_{a+b}(S)$.
3. **Inverse:** $\sigma_{-n}(\sigma_n(S)) = S$ (when $\mu + n \geq 0$).

11.2 Gauge Invariance

Definition 11.2 (Observable). $\text{Obs}(S)$ extracts the partition region structure, ignoring μ .

Theorem 11.2 (Gauge Invariance (KernelNoether.v)). $\text{Obs}(\sigma_\delta(S)) = \text{Obs}(S)$. *Shifting μ by a constant does not affect observables.*

Theorem 11.3 (Noether's Theorem (KernelNoether.v, RepresentationTheorem.v — reference-only)).

1. **Forward:** *States identical except for μ lie on the same gauge orbit. (This is direct from the definition: if all other fields match, the μ difference is the gauge shift.)*
2. **Gauge Invariance:** *`z_gauge_invariance` proves that shifting μ does not change `Observable_partition`. This is definitional: `z_gauge_shift` only modifies the `vm_mu` field, and `Observable_partition` reads only `vm_graph`.*
3. **Trace Preservation:** *Gauge-equivalent states produce identical observable traces (same labels, same μ -costs) for any finite horizon. Proven as `vm_step_orbit_equiv`.*

The μ -ledger is a gauge degree of freedom. Its absolute value is unobservable; only differences $\Delta\mu$ are physical. The “Noether” label is a deliberate analogy: the result follows from the record structure (separate fields for μ and graph), not from a deep symmetry principle.

Theorem 11.4 (μ -Monotonicity (KernelNoether.v)). $\text{vm_step}(S, i, S') \implies \mu(S) \leq \mu(S')$. *The μ -ledger never decreases under the dynamics. This is true by construction: instruction costs are natural numbers (≥ 0), so adding them to the ledger cannot decrease it. The proof examines each step constructor and confirms the arithmetic.*

Chapter 12

Two-Dimensional Amplitude Space

Remark 12.1 (Epistemological Status: (C) Conditional Derivation). The following derivation assumes that partition states admit *amplitude* representations (superpositions), not merely classical probability distributions. This is stated as Axiom 12.1. Without it, classical probability theory ($p \in [0, 1]$, one-dimensional, continuous) suffices and the 2D argument does not apply. The passage from a discrete binary partition to the continuum S^1 also requires a limiting process not formalized in the Coq suite. The argument does not rule out quaternionic (4D) amplitudes; it establishes 2D as the *minimum* dimension for continuous superposition.

Axiom 12.1 (Superposition Principle). Partition module states admit amplitude representations: a state with n classical configurations is described by n real amplitudes $(a_1, \dots, a_n)$ satisfying $\sum_i a_i^2 = 1$, where a_i^2 gives the probability of configuration i .

Given this axiom, binary partition structure forces quantum amplitudes to live in at least two dimensions.

Theorem 12.1 (1D Is Insufficient (TwoDimensionalNecessity.v — reference-only)). *A one-dimensional normalized amplitude satisfies $x^2 = 1$, hence $x \in \{+1, -1\}$. No intermediate superpositions exist.*

Theorem 12.2 (2D Is Necessary and Sufficient (TwoDimensionalNecessity.v — reference-only)). *Binary partition structure requires exactly 2D amplitude space. Two-dimensional amplitudes (a, b) with $a^2 + b^2 = 1$ admit a continuous family via $a = \cos \theta$, $b = \sin \theta$: the unit circle S^1 .*

Chapter 13

Complex Amplitudes from Norm Preservation

Zero-cost evolution must preserve the norm $a^2 + b^2 = 1$. The group of norm-preserving maps on S^1 is $\text{SO}(2) \cong U(1)$, forcing amplitudes to be complex numbers.

Theorem 13.1 (Rotation Group (ComplexNecessity.v — reference-only)). *2D rotations $R_\theta(a, b) = (a \cos \theta - b \sin \theta, a \sin \theta + b \cos \theta)$ satisfy:*

1. $|R_\theta(a, b)|^2 = a^2 + b^2$ (norm preservation).
2. $R_0 = \text{id}$ (identity).
3. $R_{\theta_1} \circ R_{\theta_2} = R_{\theta_1 + \theta_2}$ (composition = addition).
4. $R_\theta^{-1} = R_{-\theta}$ (invertibility).

Theorem 13.2 (Complex Necessity (ComplexNecessity.v — reference-only)). *Complex multiplication by $e^{i\theta}$ is exactly 2D rotation. The norm-preserving maps on S^1 are exactly complex multiplications: $\text{SO}(2) \cong U(1)$. Hence complex amplitudes are the minimal structure consistent with continuous 2D superposition; higher-dimensional extensions (quaternionic, etc.) are not ruled out by this argument alone.*

Corollary 13.3 (Euler's Formula). $e^{i\theta_1} \cdot e^{i\theta_2} = e^{i(\theta_1 + \theta_2)}$.

Derivation chain: Binary partition $\rightarrow$ 2D amplitudes $\rightarrow$ normalization (S^1) $\rightarrow$ zero-cost $\Rightarrow$ norm-preserving $\Rightarrow \text{SO}(2) \cong U(1) \Rightarrow$ complex numbers.

Chapter 14

The Born Rule

Definition 14.1 (Bloch Sphere Probabilities). For a state with purity vector (x, y, z) :

$$P(|0\rangle) = \frac{1+z}{2}, \quad P(|1\rangle) = \frac{1-z}{2}$$

Definition 14.2 (Measurement μ -Cost).

$$\text{Cost}(x, y, z) = \frac{1 - (x^2 + y^2 + z^2)}{2}$$

This is the linear entropy. For pure states ($r^2 = 1$), $\text{cost} = 0$. For mixed states ($r^2 < 1$), $\text{cost} > 0$.

Theorem 14.1 (Uniqueness of the Born Rule (BornRule.v)). *The Born rule $P(|0\rangle) = (1+z)/2$ is the **unique** probability assignment satisfying:*

1. *Non-negativity and normalization:* $P \geq 0$, $P(|0\rangle) + P(|1\rangle) = 1$.
2. *Eigenstate consistency:* $P(0, 0, 1) = 1$, $P(0, 0, -1) = 0$.
3. *Linearity in z :* $P(x, y, z) = az + b$ for some a, b .

Proof: Boundary conditions force $a = 1/2$, $b = 1/2$. No other solution exists.

Note: The Coq theorem also lists a `mu_consistent` hypothesis, but it is **unused** in the proof. The hypothesis `mu_consistent P` unfolds to “measurement cost ≥ 0 for valid Bloch vectors,” which is directly from the definition true for all states and does not constrain P . The linearity assumption (item 3) does all the work—it assumes the functional form $P = az + b$ and solves a 2-equation system. This is less “deriving the Born rule from μ -accounting” and more “the Born rule is the unique linear rule consistent with eigenstates.” The non-circular derivation from tensor consistency (`BornRuleFromSymmetry.v`, below) is the stronger result.

Theorem 14.2 (Born Rule from Tensor Consistency (BornRuleFromSymmetry.v — reference-only/replaced; active derivation in `kernel/BornRuleLinearity.v`)). *Let $g : [0, 1] \rightarrow [0, 1]$ be any function satisfying:*

1. *Eigenstate consistency:* $g(0) = 0$, $g(1) = 1$.
2. *Normalization:* $g(x) + g(1-x) = 1$ for all $x \in [0, 1]$.
3. *Tensor product consistency:* $g(xy) = g(x) \cdot g(y)$ for $x, y \in [0, 1]$.
4. *Range:* $0 \leq g(x) \leq 1$ for $x \in [0, 1]$.

Then $g(x) = x$ for all $x \in [0, 1]$.

Remark 14.1 (Epistemological Status: **(C)** Conditional Derivation — Circularity Resolved). The original proof (`BornRule.v`) depends on the linearity axiom (item 3): P is affine in z . This assumption is equivalent to the Born rule restated, raising a circularity concern.

The new derivation (`BornRuleFromSymmetry.v`) replaces linearity with *tensor product consistency*: independent measurements yield independent outcomes, i.e., $g(xy) = g(x)g(y)$. This is a structural property of the machine, not a physics axiom. It follows from module independence proven in `ThieleUnificationTensor.v`. The proof proceeds: $g = \text{id}$ on all dyadic rationals (by multiplicativity + normalization); then monotonicity + density of dyadics forces $g = \text{id}$ everywhere (via the Archimedean property of $\mathbb{R}$). Zero admits.

Chapter 15

Unitarity

Definition 15.1 (Evolution). An evolution E maps Bloch vectors $(x, y, z) \mapsto (x', y', z')$ with associated μ -cost.

Theorem 15.1 (Unitarity from Zero Cost (Unitarity.v)).

1. $\mu = 0 \implies r'^2 \geq r^2$ (purity cannot decrease at zero cost). Proven as `zero_cost_preserves_purity`.
2. Non-unitary evolution ($r'^2 < r^2$ for some state) requires $\mu > 0$. Proven as `nonunitary_requires_mu`: the proof specializes the conservation hypothesis at the witness, then `lra`.
3. $\mu = 0 \implies$ no info loss (one direction proven). The converse and the full equivalence $\mu = 0 \Leftrightarrow$ unitary $\Leftrightarrow$ reversible are stated in the file but not proven as theorems: they appear as *Definition* declarations (propositional constants) rather than proven *Theorems*.

Theorem 15.2 (Lindblad Requires μ (Unitarity.v)). Lindblad-type dissipation at rate $\gamma > 0$ requires $\mu \geq \gamma$. Decoherence is paid for.

Theorem 15.3 (CPTP Structure (Unitarity.v)). Positivity + trace preservation $\implies$ the evolution is CPTP (completely positive, trace-preserving).

Chapter 16

Purification

Theorem 16.1 (Purification Principle (Purification.v)). *Every mixed state (x, y, z) with $r^2 = x^2 + y^2 + z^2 \leq 1$ admits eigenvalues λ_1, λ_2 with:*

- $\lambda_1 + \lambda_2 = 1, \quad 0 \leq \lambda_i \leq 1.$
- $(\lambda_1 - \lambda_2)^2 = r^2.$
- *Construction:* $\lambda_1 = (1 + r)/2, \lambda_2 = (1 - r)/2.$

Pure states ($r = 1$) need no external reference (deficit = 0). The maximally mixed state ($r = 0$) has deficit = 1.

Chapter 17

No-Cloning

Theorem 17.1 (No-Cloning from μ -Conservation (NoCloning.v)). *Let $I = x^2 + y^2 + z^2$ be the information content (purity). For a cloning operation with outputs of information I_1, I_2 and μ -cost μ :*

$$I_1 + I_2 \leq I + \mu$$

Theorem 17.2 (No-Cloning from μ -Monotonicity (NoCloningFromMuMonotonicity.v — reference-only/replaced; active proof in `kernel/NoCloning.v`)). *Machine-native no-cloning using nat-valued structural content (MDL complexity). If cloning duplicates n bits of structural content while producing zero new bits: $2n \leq n + 0$ implies $n \leq 0$. The entire proof is a single `lia` step (linear integer arithmetic). No Bloch sphere, no Hilbert space, no real analysis. Zero admits.*

Corollary 17.3 (Perfect Cloning Is Impossible at Zero Cost). *$I_1 = I_2 = I$ and $\mu = 0$ implies $2I \leq I$, hence $I \leq 0$. Only the trivial state can be cloned for free.*

Corollary 17.4 (Cloning Cost). *Perfect cloning requires $\mu \geq I$. For pure states: $\mu \geq 1$.*

Theorem 17.5 (Approximate Cloning (NoCloning.v)). *For fidelities f_1, f_2 : $f_1 + f_2 \leq 1 + \mu/I$. At zero cost: $f_1 + f_2 \leq 1$.*

Theorem 17.6 (No Deletion (NoCloning.v)). *Perfect deletion also requires $\mu \geq I$ (dual of no-cloning).*

Chapter 18

Observation and Collapse

18.1 Observation Irreversibility

Theorem 18.1 (REVEAL Is Irreversible (ObservationIrreversibility.v — reference-only)). *For bits > 0 : $\mu_{\text{after}} > \mu_{\text{before}}$, and the post-REVEAL state $\neq$ the pre-REVEAL state. Observation prevents recovery of the pre-measurement superposition.*

18.2 Collapse Determination

Definition 18.1 (Partition Entropy). $H(P) = \log_2(\text{dim}(P))$ where dim is the partition state dimension.

Theorem 18.2 (Maximum Information Implies Unique State (CollapseDetermination.v — reference-only)). *If bits revealed $= H(P_{\text{before}})$ (maximum information), then $\text{dim}(P_{\text{after}}) = 1$. The measurement fully collapses the state to a unique outcome. This is the **projection postulate**, derived rather than assumed.*

Chapter 19

The Tsirelson Bound

Definition 19.1 (Correlation μ -Cost). $\mu_{\text{corr}}(S) = 0$ if $|S| \leq 2\sqrt{2}$; otherwise $\mu_{\text{corr}} = |S| - 2\sqrt{2}$.

Remark 19.1 (Epistemological Status: **(C)** Conditional Derivation — Circularity Resolved). The original Coq proof (`TsirelsonDerivation.v` — reference-only/replaced) encoded $2\sqrt{2}$ in the definition of μ_{corr} , making its derivation status **(R)**.

This gap is now closed. `TsirelsonGeneral.v` derives $S^2 \leq 8$ from pure algebra: a sum-of-squares identity proves each CHSH row contributes $\leq 2(e_1^2 + e_2^2)$, then Cauchy–Schwarz bounds the sum by $4 \sum e_i^2 \leq 8$. `TsirelsonFromAlgebra.v` provides a self-contained algebraic derivation with achievability witness ($e = \pm 1/\sqrt{2}$, giving $S = 2\sqrt{2}$) and a verified rational bound ($\sqrt{8} < 5657/2000$). Zero admits in both files.

Theorem 19.1 (Tsirelson from Zero μ (`TsirelsonDerivation.v` — reference-only/replaced) — **(R)**). *Total $\mu = 0$ implies $|S_{\text{CHSH}}| \leq 2\sqrt{2} \approx 2.828$.*

Theorem 19.2 (CHSH Separation (`Deliverable_CHSHSeparation.v` — reference-only)). *Strict numerical separation:*

$$2 < 2\sqrt{2} \approx 2.828 < \frac{16}{5} = 3.2$$

- **Classical** (local receipts): $|S| \leq 2$ (Bell bound).
- **Quantum** (admissible, $\mu = 0$): $|S| \leq 2\sqrt{2}$ (Tsirelson bound).
- **Supra-quantum** ($\mu > 0$ allowed): witnesses achieve $|S| = 3.2$.

The Tsirelson bound $2\sqrt{2}$ is the maximum correlation purchasable at zero μ -cost. Exceeding it requires paying for additional structural information.

Chapter 20

Witness Insight Taxonomy

Theorem 20.1 (CHSH Trial Is Tier 0 (kernel/WitnessInsightGeneral.v) — (S)). *chsh_trial_is_tier0: The CHSH_TRIAL instruction never touches either certification channel:*

- *chsh_trial_not_cert_addr_setter: CHSH_TRIAL does not change csr_cert_addr.*
- *chsh_trial_preserves_vm_certified: CHSH_TRIAL does not change vm_certified.*

Recording CHSH trial outcomes is provably free of structural cost. The counters go up; the cert channels are untouched. Cost: 0. Zero Admitted.

Theorem 20.2 (Three-Tier Observation Taxonomy (kernel/WitnessInsightGeneral.v) — (S)). *witness_insight_complete_taxonomy: Every observation the machine can make falls into exactly one of three tiers:*

Tier 0 *Raw observation (always free): Records data without activating any certification channel. CHSH_TRIAL is the canonical example. Cost: $\Delta\mu = 0$ is possible; no cert channel is touched.*

Tier 1 *Certified structural insight (always costs ≥ 1): Any instruction that activates `csr_cert_addr` or `vm_certified` is in this tier. Proven: `certified_witness_insight_nonfree` shows μ increases by at least 1.*

Tier 2 *Certified non-local witness insight (always costs ≥ 1): A certified execution whose `WitnessCount` statistics show $S > 2$. `nonlocal_witness_insight_nonfree`: getting certified with non-local evidence requires $\mu \geq 1$.*

witness_insight_nonfree_general: For any instruction in Tier 1 or Tier 2, μ increases by at least 1. The only free observations are Tier 0. Zero Admitted.

Chapter 21

Bell's Theorem in Morphism Coupling Language

Definition 21.1 (Setting-Outcome Coupling (kernel/CHSHCouplingBridge.v)). The `chsh_setting_outcome_c` maps a `WitnessCount` record to a coupling (list of $(nat \times nat)$ pairs). Each pair records a (setting-pair-index, outcome-type) observation. A coupling is **separable** (functional) when it maps each setting to at most one outcome type; **non-separable** otherwise.

Theorem 21.1 (Locally Consistent Strategy Gives Separable Coupling (kernel/CHSHCouplingBridge.v) — (S)). *locally_consistent_gives_separable_coupling: Any WitnessCount configuration produced by a locally deterministic strategy $(a_0, a_1, b_0, b_1) \in \{0, 1\}^4$ gives a separable (functional) coupling. Zero Admitted.*

Theorem 21.2 (Local Coupling Bound (kernel/CHSHCouplingBridge.v) — (S)). *locally_consistent_classic: Any locally factorizable morphism coupling satisfying `WCLocallyConsistent` has $|S| \leq 2$. Bell's theorem in the morphism coupling language: the separation between local and non-local correlations is not just a statistical fact but a structural property of the coupling predicate. Zero Admitted.*

Theorem 21.3 (CHSH Violation Rules Out Local Couplings (kernel/CHSHCouplingBridge.v) — (S)). *chsh_violation_rules_out_locally_factorizable_coupling: If WitnessCount statistics satisfy $S > 2$, no locally factorizable morphism coupling is consistent with those counts. Contrapositively: non-local correlations cannot be produced by any local morphism coupling. The no-go is stated in the coupling language, not just the statistical layer. Zero Admitted.*

Remark 21.1. These results connect the categorical morphism layer directly to Bell nonlocality. A claim that two modules are entangled (via `MORPH_ASSERT` on a non-separable coupling) cannot be locally faked: the NoFI law charges at least 1 for the `MORPH_ASSERT`, and the coupling structure itself rules out any local hidden strategy that would produce $S > 2$ for free.

Chapter 22

The Schrödinger Equation

Theorem 22.1 (Emergent Schrödinger (EmergentSchrodinger.v — reference-only)). *The finite-difference Schrödinger equation emerges from the partition update rules. For a two-component amplitude state (a, b) with mass m , potential V , and time step Δt :*

$$a(t + \Delta t) = a - \frac{\Delta t}{2m} \nabla^2 b + \Delta t \cdot V b \quad (22.1)$$

$$b(t + \Delta t) = b + \frac{\Delta t}{2m} \nabla^2 a - \Delta t \cdot V a \quad (22.2)$$

*The coupling is **antisymmetric**: $c_{a,\nabla^2 b} = -c_{b,\nabla^2 a}$ and $c_{a,Vb} = -c_{b,Va}$, which is necessary and sufficient for probability conservation $\partial_t(a^2 + b^2) = 0$.*

Remark 22.1 (Epistemological Status: **(C)** Conditional Derivation). This is exactly the finite-difference discretization of $i\hbar\partial_t\psi = -\frac{\hbar^2}{2m}\nabla^2\psi + V\psi$ with $\psi = a + ib$. The Schrödinger form emerges *conditionally*: given Axiom 12.1 (which entails 2D amplitudes) and the requirement that zero-cost evolution preserves probability ($\partial_t(a^2 + b^2) = 0$), the antisymmetric coupling structure is the unique linear solution. The mass m and potential V are parameters, not derived.

Chapter 23

Planck's Constant

Remark 23.1 (Epistemological Status: **(R)** Consistency Relation). The following is **not** a derivation of Planck's constant from first principles. It is a consistency relation that gives h a physical interpretation within the μ -framework. The Coq proof verifies an algebraic identity; the physics is in the definitions.

Definition 23.1 (Physical Constants (Normalized Units)). $k_B = 1/100$, $T = 1$, $h = 1$. Landauer energy: $E_L := k_B T \ln 2$.

Definition 23.2 (Computational Time Step). $\tau_\mu := h/(4E_L)$, the Margolus–Levitin time at Landauer energy.

Theorem 23.1 (Planck Consistency Relation (PlanckDerivation.v — reference-only) — **(R)**).

$$h = 4 \cdot E_L \cdot \tau_\mu$$

Remark 23.2 (Why This Is Circular). Substituting $\tau_\mu = h/(4E_L)$ yields $h = 4E_L \cdot h/(4E_L) = h$. The Coq proof reduces to the `field` tactic (pure algebra). This does *not* predict h ; it defines τ_μ in terms of h and verifies consistency.

What the relation does provide: a physical interpretation of Planck's constant as $4 \times$ the action (energy $\times$ time) of one μ -bit operation at Landauer cost. If taken as a physical claim, the testable content is the predicted time step $\tau_\mu = h/(4k_B T \ln 2)$, which at room temperature yields $\tau_\mu \approx 5.7 \times 10^{-14}$ s (femtosecond range, consistent with molecular vibration timescales).

What would make this non-circular: defining τ_μ as an independent primitive (measured experimentally) and then *deriving* $h = 4E_L \tau_\mu$ as a *prediction*. The current formalization does not do this.

Chapter 24

Emergent Spacetime

24.1 The μ -Metric

Definition 24.1 (μ -Distance (MuGeometry.v)). The Coq formalization defines `mu_distance` as the absolute difference of μ -totals:

$$d_\mu(S_1, S_2) = |\mu_{\text{total}}(S_2) - \mu_{\text{total}}(S_1)|$$

This is a one-dimensional projection, not a minimum over paths.

Theorem 24.1 (Pseudometric Properties (MuGeometry.v)). *d_μ satisfies non-negativity, reflexivity ($d(S, S) = 0$), symmetry, and triangle inequality, all inherited from absolute value on $\mathbb{Z}$. Note: d_μ is a **pseudometric**, not a metric: $d(S_1, S_2) = 0$ does not imply $S_1 = S_2$ (many distinct states share the same μ -total). The “geometry” is a 1D projection onto the μ counter.*

24.2 Causal Cones

Definition 24.2 (μ -Reachability Cone (SpacetimeEmergence.v)). $C^+(S) = \{S' \mid \exists \tau. S \xrightarrow{\tau} S' \wedge \text{cost}(\tau) \leq \text{Budget}\}$.

This is a structural analogy to causal cones: the set of reachable states given a finite μ -budget plays a role analogous to a light cone. The analogy should not be over-read: there is no Lorentz invariance, no metric signature, and no boost transformations in the Coq formalization.

24.3 No-Signaling

Theorem 24.2 (Observational Locality (SpacetimeEmergence.v)). *If module M_B is not in the target set of any instruction in a trace, then M_B ’s observable region is unchanged after execution (`exec_trace_no_signaling_outside_cone`).*

Remark 24.1. This is a **data isolation** property: operations targeting module A in the partition graph do not affect the lookup of module B . The proof is structural (list operations on disjoint keys), non-trivial mainly for `psplit/pmerge` which restructure the graph. The label “no-signaling” is an analogy—there is no spatial metric or speed limit between modules.

24.4 Four-Dimensional Spacetime Geometry

The previous sections treated spacetime as a 1D projection via the μ -metric. The next sections extend to full 4D geometry with discrete differential structure, building discrete differential geometry from computational primitives.

24.4.1 4D Simplicial Complex

Definition 24.3 (4D Simplicial Complex (FourDSimplicialComplex.v)). A **4D simplicial complex** $\mathcal{K}$ is a discrete approximation to 4D spacetime consisting of:

- **Vertices** V : Computational modules (ModuleID)
- **Edges** $E \subseteq V \times V$: Connections between modules
- **Simplices**: 2-simplices (triangles), 3-simplices (tetrahedra), 4-simplices (4D cells)

The complex satisfies:

- **Consistency**: Every face of a simplex is also in $\mathcal{K}$
- **Well-formedness**: Dimension 4 with locally Euclidean structure

24.4.2 Metric Tensor from μ -Costs

Definition 24.4 (Computational Metric (MetricFromMuCosts.v, RiemannTensor4D.v)). The metric tensor at vertex v with respect to vertices w_1, w_2 is:

$$g_{\mu\nu}(v; w_1, w_2) = \begin{cases} \ell(v, w_1, w_2) & \text{if } \mu = \nu \\ 0 & \text{if } \mu \neq \nu \end{cases} \quad (24.1)$$

where the edge length function is:

$$\ell(s, v_1, v_2) = 2 \cdot \text{mass}(v_1) \quad \text{if } v_1 = v_2 \quad (24.2)$$

and mass is derived from structural complexity: $\text{mass}(v) = |\text{module_structural_mass}(s, v)|$.

Remark 24.2. This metric is **position-dependent** for non-uniform mass distributions:

- **Uniform mass** m : $g_{\mu\mu}(v; w, w) = 2m$ (constant) $\implies$ flat spacetime
- **Non-uniform mass**: $g_{\mu\mu}(v; w, w)$ varies with $w \implies$ curved spacetime

In this model, mass distributions create position-dependent geometry: uniform mass gives flat spacetime, non-uniform mass gives curved spacetime.

24.4.3 Discrete Differential Geometry

Definition 24.5 (Discrete Derivative (RiemannTensor4D.v)). For a scalar field $f : V \rightarrow \mathbb{R}$ and direction μ :

$$\partial_\mu f(v) = \frac{1}{|N_\mu(v)|} \sum_{w \in N_\mu(v)} (f(w) - f(v)) \quad (24.3)$$

where $N_\mu(v)$ is the set of neighbors of v in the μ -direction.

Definition 24.6 (Christoffel Symbols (RiemannTensor4D.v)). The connection coefficients (Christoffel symbols of the second kind) are:

$$\Gamma_{\mu\nu}^\rho(v) = \frac{1}{2} \sum_\sigma g^{\rho\sigma}(v) \left(\partial_\mu g_{\nu\sigma}(v) + \partial_\nu g_{\mu\sigma}(v) - \partial_\sigma g_{\mu\nu}(v) \right) \quad (24.4)$$

where $g^{\rho\sigma}$ is the inverse metric (diagonal in this construction).

Lemma 24.3 (Flat Spacetime Has Zero Connection (EinsteinEquations4D.v) — (S)). *For uniform mass distribution:*

$$\forall w, \text{mass}(w) = m \implies \Gamma_{\mu\nu}^\rho(v) = 0 \quad (24.5)$$

Proof. Uniform mass $\implies$ position-independent metric $\implies \partial_\mu g_{\nu\sigma} = 0 \implies \Gamma_{\mu\nu}^\rho = 0$. $\square$

24.4.4 Curvature Tensors

Definition 24.7 (Riemann Curvature Tensor (RiemannTensor4D.v)). The Riemann curvature tensor measures the failure of parallel transport:

$$R_{\sigma\mu\nu}^{\rho}(v) = \partial_{\mu}\Gamma_{\nu\sigma}^{\rho}(v) - \partial_{\nu}\Gamma_{\mu\sigma}^{\rho}(v) + \sum_{\lambda} \left(\Gamma_{\mu\lambda}^{\rho}(v)\Gamma_{\nu\sigma}^{\lambda}(v) - \Gamma_{\nu\lambda}^{\rho}(v)\Gamma_{\mu\sigma}^{\lambda}(v) \right) \quad (24.6)$$

Definition 24.8 (Ricci Tensor (RiemannTensor4D.v)). Contraction of the Riemann tensor:

$$R_{\mu\nu}(v) = \sum_{\rho} R_{\mu\rho\nu}^{\rho}(v) \quad (24.7)$$

Definition 24.9 (Ricci Scalar (RiemannTensor4D.v)). Complete contraction:

$$R(v) = \sum_{\mu,\nu} g^{\mu\nu}(v) R_{\mu\nu}(v) \quad (24.8)$$

Definition 24.10 (Einstein Tensor (RiemannTensor4D.v)). The Einstein tensor encodes space-time curvature:

$$G_{\mu\nu}(v) = R_{\mu\nu}(v) - \frac{1}{2}g_{\mu\nu}(v)R(v) \quad (24.9)$$

Lemma 24.4 (Flat Spacetime Has Zero Curvature (EinsteinEquations4D.v) — (S)). *For uniform mass:*

$$\forall w, \text{ mass}(w) = m \implies R_{\sigma\mu\nu}^{\rho}(v) = 0 \implies G_{\mu\nu}(v) = 0 \quad (24.10)$$

24.4.5 Stress-Energy Tensor

Definition 24.11 (Energy Density (EinsteinEquations4D.v)). The energy density is the computational mass:

$$T_{00}(v) = \text{mass}(v) \quad (24.11)$$

Definition 24.12 (Momentum Density (EinsteinEquations4D.v)). Spatial derivatives of energy:

$$T_{0i}(v) = T_{i0}(v) = \partial_i T_{00}(v) \quad (24.12)$$

Definition 24.13 (Stress Components (EinsteinEquations4D.v)). Spatial stress tensor:

$$T_{ij}(v) = \begin{cases} \text{mass}(v) & \text{if } i = j \\ 0 & \text{if } i \neq j \end{cases} \quad (24.13)$$

Definition 24.14 (Stress-Energy Tensor (EinsteinEquations4D.v)). Combining all components:

$$T_{\mu\nu}(s, \mathcal{K}, v) = \begin{cases} T_{00}(v) & \text{if } \mu = \nu = 0 \\ T_{0i}(v) & \text{if } \mu = 0, \nu = i \text{ or } \mu = i, \nu = 0 \\ T_{ij}(v) & \text{if } \mu = i, \nu = j \\ 0 & \text{otherwise} \end{cases} \quad (24.14)$$

24.4.6 Einstein's Field Equations

Definition 24.15 (Gravitational Constant (EinsteinEquations4D.v)). In computational units where the fundamental scale is set by μ -costs:

$$G = \frac{1}{8\pi} \quad (24.15)$$

This makes $8\pi G = 1$, normalizing the coupling in Einstein's equations.

Lemma 24.5 (Curvature from μ -Gradients (EinsteinEquations4D.v) — (S)). *For uniform mass distribution:*

$$\forall w, \text{ mass}(w) = m \implies G_{\mu\nu}(v) = 0 \quad (24.16)$$

Proof. The proof chain is:

1. Uniform mass $\implies$ position-independent metric
2. Position-independent metric $\implies$ zero discrete derivatives $\partial_\mu g_{\nu\sigma} = 0$
3. Zero metric derivatives $\implies$ zero Christoffel symbols $\Gamma_{\mu\nu}^\rho = 0$
4. Zero Christoffel $\implies$ zero Riemann tensor $R_{\sigma\mu\nu}^\rho = 0$
5. Zero Riemann $\implies$ zero Ricci tensor and scalar
6. Zero Ricci $\implies$ zero Einstein tensor $G_{\mu\nu} = 0$

All steps are proven constructively in EinsteinEquations4D.v. □

Lemma 24.6 (Vacuum Implies Zero Stress-Energy (EinsteinEquations4D.v) — (S)). *For vacuum states (zero mass everywhere):*

$$\forall w, \text{ mass}(w) = 0 \implies T_{\mu\nu}(v) = 0 \quad (24.17)$$

Proof. Direct computation:

- $T_{00} = \text{mass} = 0$
- $T_{0i} = \partial_i(\text{mass}) = \partial_i(0) = 0$
- $T_{ij} = \delta_{ij} \cdot \text{mass} = \delta_{ij} \cdot 0 = 0$

□

Theorem 24.7 (Einstein Equations in Vacuum (EinsteinEquations4D.v) — (S)). *For any VM state s , 4D simplicial complex $\mathcal{K}$, spacetime indices μ, ν , and vertex v :*

$$\boxed{\forall w, \text{ mass}(w) = 0 \implies G_{\mu\nu}(s, \mathcal{K}, v) = 8\pi G T_{\mu\nu}(s, \mathcal{K}, v)} \quad (24.18)$$

Note: This is the vacuum case where both sides equal zero ($0 = 0$). The gravitational constant $G := 1/(8\pi)$ is a **unit choice** (a definition), not a derived quantity. In vacuum, all masses vanish, so $T_{\mu\nu} = 0$ and uniform-mass (zero) yields $G_{\mu\nu} = 0$; the equation reduces to $0 = 8\pi \cdot \frac{1}{8\pi} \cdot 0 = 0$. The non-vacuum case is **not proven** and would require discrete Bianchi identities not present in the formalization.

Proof. **Step 1:** Expand the right-hand side using $G = 1/(8\pi)$:

$$8\pi G T_{\mu\nu} = 8\pi \cdot \frac{1}{8\pi} \cdot T_{\mu\nu} = T_{\mu\nu} \quad (24.19)$$

Step 2: Apply Lemma 24.5 (curvature from μ -gradients):

Vacuum (mass = 0 everywhere) is a special case of uniform mass distribution, so:

$$G_{\mu\nu}(v) = 0 \quad (24.20)$$

Step 3: Apply Lemma 24.6 (stress-energy conservation):

For vacuum:

$$T_{\mu\nu}(v) = 0 \quad (24.21)$$

Conclusion: Both sides equal zero:

$$0 = 0 \quad \checkmark \quad (24.22)$$

The complete proof is formalized in `coq/kernel/EinsteinEquations4D.v` with:

- No project-local axioms
- Zero admitted lemmas
- Zero classical logic imports
- Verified by automated proof auditing (Inquisitor: 0 HIGH findings)

□

Remark 24.3 (Vacuum vs. Non-Vacuum). Theorem 24.7 requires the vacuum hypothesis (mass = 0 everywhere). For non-vacuum states with arbitrary mass distributions, the equation $G_{\mu\nu} = 8\pi G T_{\mu\nu}$ only holds when the distribution satisfies discrete Bianchi identities (conservation laws). This is analogous to classical general relativity, where Einstein's equations are *field equations* that constrain which configurations are physically possible, not universal identities satisfied by all states.

The vacuum case is sufficient to demonstrate that spacetime curvature emerges from computational dynamics. The non-vacuum case requires discrete differential geometry beyond the current formalization (specifically, discrete covariant derivatives and discrete Bianchi identities).

Remark 24.4 (Epistemological Status). This is a **(S) Structural** result: it proves a mathematical relationship between computational primitives (VM states, μ -costs) and geometric quantities (curvature tensors) with no physical axioms. The theorem establishes that:

1. Spacetime geometry (metric, curvature) can be constructed from pure computation
2. The resulting geometry satisfies the vacuum Einstein equation in the formally verified zero-mass regime
3. Beyond that vacuum regime, the repository contains a conditional full-tensor bridge with named premises, a proved uniform diagonal boundary witness in the active operator, and a separately proved non-uniform boundary witness in a concrete affine metric-scaled redesign branch rather than one universal unconditional theorem

The result does *not* claim that physical spacetime *is* implemented by a Turing machine—that would be a (C) conditional claim requiring empirical validation. More importantly, it does not justify the stronger statement that arbitrary computational substrates with μ -cost tracking automatically satisfy the full Einstein equations in complete generality. The honest status is narrower: the vacuum theorem is unconditional; the full-tensor bridge in `EinsteinEquationsFull.v` remains conditional; the active first-neighbor operator proves the uniform diagonal boundary witness and formally refutes the stronger non-uniform diagonal extension; the broader non-negative local weighted family containing the simple symmetric average also fails on the same shell; and `SymmetricDerivative4D.v` proves that a concrete signed affine metric-scaled redesign rescues that non-uniform boundary witness with off-diagonal Ricci vanishing, Einstein tensor $G_{\mu\nu} = 3\delta_{\mu\nu}$ at the critical vertex, and, under `module_structural_mass s 1 = 1`, the aligned full-tensor witness $G_{\mu\nu} = 3T_{\mu\nu}$ there.

Corollary 24.8 (Discrete Vacuum Geometry is Emergent). *Discrete vacuum spacetime geometry is an emergent property of the machine’s information-processing structure.*

Proof. Theorem 24.7 constructs the discrete metric, curvature, and vacuum Einstein relation from computational primitives (VM states, operations, μ -costs) with no physical axioms. Therefore, this model’s discrete vacuum geometry is a derived concept rather than a primitive one. $\square$

24.4.7 Empirical Validation

The theoretical results are validated numerically:

Theorem 24.9 (Numerical Verification (tests/test_*) — Empirical).

1. **Flat spacetime** (uniform mass $m = 1$):

- $\max |\Gamma_{\mu\nu}^\rho| < 10^{-10} \checkmark$
- $\max |R_{\sigma\mu\nu}^\rho| < 10^{-10} \checkmark$
- $\max |G_{\mu\nu}| < 10^{-10} \checkmark$

2. **Curved spacetime** (non-uniform mass $m \in [0.5, 1.5]$):

- $\max |\Gamma_{\mu\nu}^\rho| \approx 3.0 \checkmark$ (curvature detected)
- $\max |R_{\sigma\mu\nu}^\rho| > 0 \checkmark$

3. **Vacuum Einstein equations** (mass = 0):

- $|G_{\mu\nu}| < 10^{-10} \checkmark$
- $|T_{\mu\nu}| < 10^{-10} \checkmark$
- $|G_{\mu\nu} - 8\pi GT_{\mu\nu}| < 10^{-10} \checkmark$

These tests confirm that the discrete differential geometry correctly implements the theoretical predictions.

24.4.8 Affine Metric-Scaled Operator: Non-Uniform Boundary Results

The first-neighbor discrete operator refutes the non-uniform diagonal extension of the EFE. A signed affine metric-scaled redesign closes the gap at vertex 1 and fully characterizes the outer vertices.

Theorem 24.10 (Affine Off-Diagonal Ricci Zero (kernel/AffineEFEClosure.v) — (S)). *affine_off_diagonal_r*
Under the affine metric-scaled symmetric operator, with the non-uniform isotropic boundary 4-simplex metric, off-diagonal Ricci vanishes at vertex 1: $R_{\mu\nu}(v_1) = 0$ for $\mu \neq \nu$. Zero Admitted. Zero Section Variables.

Theorem 24.11 (Full Tensor EFE at Vertex 1 (kernel/AffineEFEClosure.v) — (S)). *affine_full_tensor_efe*. Under *module_structural_mass s 1 = 1* (unit structural mass at vertex 1), the full tensor Einstein field equation holds:

$$G_{\mu\nu}(v_1) = 3 \cdot T_{\mu\nu}(v_1)$$

for all $\mu, \nu \in \{0, 1, 2, 3\}$. Zero Admitted.

Theorem 24.12 (Outer Vertex Characterization (kernel/AffineEFEClosure.v) — (S)). At the outer vertices $v \in \{0, 2, 3, 4\}$ of the boundary 4-simplex, the affine operator produces:

- *affine_ricci_offdiag_outer*: Off-diagonal Ricci = $3/32$ (non-zero).
- *affine_ricci_diag_outer*: Diagonal Ricci = $-45/32$.
- *affine_ricci_scalar_outer*: Ricci scalar = $-45/8$.
- *affine_einstein_outer*: Einstein tensor = $45/32$ (diagonal) and $3/32$ (off-diagonal).
- *affine_efc_fails_outer_offdiag*: The diagonal EFE form $G = 3T$ fails at outer vertices because G has non-zero off-diagonal components.

The affine factor $8 - 5g_{00}$ is specifically tuned to the center vertex geometry; at outer vertices it produces a non-trivial off-diagonal Einstein tensor. Zero Admitted.

Remark 24.5 (What Remains Open). The affine operator closes vertex 1 and characterizes the outer vertices. The remaining open question is whether the affine factor $8 - 5g_{00}$ is the correct physical discretization. That requires connecting the discrete operator to a physical substrate — not a Coq question, an empirical one.

Chapter 25

Information Causality

Theorem 25.1 (IC- μ Equivalence (InformationCausality.v) — (R)). *The file defines two record types (`ICScenario` and `MuScenario`) with opaque `Prop` fields, then proves they are equivalent given a hypothesis (`ic_mu_equivalent`) that literally states the equivalence. The proof is *tauto*. No mutual information, probability distributions, or CHSH values appear anywhere in the formalization.*

Remark 25.1. The intended statement — that Bob’s total information about Alice’s data is bounded by the channel’s μ -capacity — is a reasonable physical claim but is **not formalized** in the Coq file. The current file is a placeholder that assumes the conclusion as a hypothesis and re-derives it. A future version would need to define entropy, mutual information, and prove the bound from the VM semantics.

Chapter 26

Thermodynamic Bridge

26.1 Landauer’s Principle

Theorem 26.1 (Landauer Derived (LandauerDerived.v)).

1. *erasure_irreversible*: Erasing ≥ 1 bit implies fan-in = $2^k > 1$ (arithmetic: $2^k > 1$ for $k \geq 1$).
2. *erasure_decreases_entropy*: Erasure decreases system entropy ($\Delta S < 0$) — this is just output < input $\Rightarrow$ output – input < 0.
3. *landauer_information_bound*: For any physical erasure, environment entropy increase $\geq$ bits erased.
4. *erasure_additive*: Sequential erasures compose additively.

*Caveat on item 3: The `PhysicalErasure` record has `second_law_satisfied` as a **field**, not a derived property. The theorem `landauer_information_bound` simply extracts this field, making the bound an input assumption rather than a derivation. The physical energy bound $Q \geq k_B T \ln 2 \cdot n$ is **explicitly not proven** (the file itself notes the conversion factor is not formalized). Also, the `mu` in this file is a standalone synonym for `bits_erased`—it has no formal connection to `vm_mu` in `VMState.v`.*

26.2 Dissipative Embedding

Theorem 26.2 (Dissipative Model (DissipativeEmbedding.v — reference-only)). *Irreversible (dissipative) physics embeds into the Thiele VM with $\Delta\mu \geq 1$ per irreversible step. Reversible physics embeds with $\Delta\mu = 0$.*

Theorem 26.3 (Reversible Physics (PhysicsEmbedding.v — reference-only, WaveEmbedding.v — reference-only)). *Reversible lattice gas and wave models embed with $\mu_{\text{final}} = \mu_{\text{init}}$ (zero cost). Particle count, momentum, wave energy, and wave momentum are conserved by the VM.*

Chapter 27

Self-Reference and the Thiele Manifold

27.1 Self-Reference Escalation

Theorem 27.1 (Gödel-Style Escalation (SelfReference.v)). *Any self-referential system S requires a meta-system Meta with strictly more dimensions:*

$$\text{self_ref}(S) \implies \exists \text{Meta} : \dim(\text{Meta}) > \dim(S)$$

The meta-system inherits self-reference, creating an infinite escalation.

27.2 The Thiele Manifold

Definition 27.1 (Thiele Manifold (ThieleManifold.v)). *An infinite tower of systems $\{L_n\}_{n \in \mathbb{N}}$ with:*

- $\dim(L_n) = 4 + n$ (level n has dimension $4 + n$).
- $\dim(L_{n+1}) > \dim(L_n)$ (strict enrichment).
- Each level reasons about the level below.
- Self-reference at level n is answered by level $n + 1$.

Theorem 27.2 (Projection to Spacetime (ThieleManifold.v)). *The projection π_4 collapses the tower to dimension 4 (spacetime). For $n > 0$:*

- π_4 is lossy: $\dim(L_n) > 4$.
- $\mu\text{-cost of projection} = \dim(L_n) - 4 = n > 0$.

Spacetime is the “shadow” of the full manifold.

27.3 Genesis: Process–Machine Isomorphism

Theorem 27.3 (Genesis (Genesis.v — reference-only)). *There is a definitional isomorphism $\text{Proc} \cong \text{Thiele}$ between coherent processes (step + admissibility proof) and Thiele machines:*

$$\text{thiele_to_proc}(\text{proc_to_thiele}(P)) = P \tag{27.1}$$

$$\text{proc_to_thiele}(\text{thiele_to_proc}(T, H)) = T \tag{27.2}$$

Any coherent process is a Thiele machine, and vice versa.

Chapter 28

Three-Layer Isomorphism

Theorem 28.1 (Observable Comparison Contract (ThreeLayerIsomorphism.v)). *The active Coq theorem packages the observable-state contract used for cross-layer comparison: if the chosen Python and RTL decoders satisfy the same projection contract as the Coq layer, then the decoded observable states agree on that projection for a shared trace τ .*

$$\text{decode}_{\text{Python}}(\tau) = \text{decode}_{\text{Coq}}(\tau) \wedge \text{decode}_{\text{RTL}}(\tau) = \text{decode}_{\text{Coq}}(\tau) \implies \text{decode}_{\text{Python}}(\tau) = \text{decode}_{\text{RTL}}(\tau)$$

This is a contract theorem over explicit observable projections, not an unconditional claim that every current Python and RTL artifact is universally equal to Coq in all CI environments.

28.1 RTL Coverage: Zero Gaps

Theorem 28.2 (RTL Gap Registry Empty (kami_hw/RTLGapRegistry.v) — (S)). *`rtl_gap_count`: The RTL gap registry is empty. There are zero unresolved coverage gaps between the Kami hardware specification and the Coq kernel semantics:*

$$\text{rtl_gap_count} = 0$$

`rtl_coverage_partition`: The 46-opcode ISA partitions as:

$$36 \text{ (unconditional Qed)} + 10 \text{ (conditional Qed under inductive invariants)} + 0 \text{ (gaps)} = 46$$

This is a Coq theorem, not a comment. The partition equation is machine-checked. Zero Admitted.

Remark 28.1. The 36 unconditional proofs cover opcodes through `SupportedOpcode`, plus `CALL`, `RET`, `CHSH_TRIAL` (`EmbedStep_wf.v`), `TENSOR_SET`, `TENSOR_GET`, `LASSERT` (`GraphReconstructionBridge.v`). The 10 conditional proofs cover the 4 partition opcodes and 7 morphism opcodes under `extended_hw_invariant` / `coupling_wf`, which are proved inductive: they hold at initialization and are preserved by every instruction in the ISA.

Chapter 29

The Curry–Howard–Thiele Correspondence

Theorem 29.1 (Logic–Computation Isomorphism (LogicIsomorphism.v — reference-only)). *The Thiele Machine extends the Curry–Howard correspondence:*

<i>Logic</i>	<i>Computation</i>	<i>Thiele</i>
<i>Proposition</i>	<i>Type</i>	<i>Partition</i>
<i>Proof</i>	<i>Program (term)</i>	<i>Trace</i>
<i>Cut elimination</i>	<i>β-reduction</i>	<i>Execution (Run)</i>
<i>Proof equivalence</i>	<i>$=_\beta$</i>	<i>Execution equivalence</i>

A valid proof corresponds to a terminating execution. Proof equivalence $\Leftrightarrow$ execution equivalence.

Theorem 29.2 (Logic Is Physics (LogicToPhysics.v — reference-only)). *Cut elimination in logic = relational composition in physics:*

$$\text{interp}(\text{cut}(\pi_1, \pi_2)) = \text{rel_comp}(\text{interp}(\pi_1), \text{interp}(\pi_2))$$

Logical proof composition maps to physical relation composition. This is the formal nucleus of the claim that logic and physics share the same categorical structure.

Chapter 30

Categorical Structure

30.1 The Morphism Layer

The active implementation adds a concrete category directly to the machine state. This is not an abstract analogy — it is an executable layer of 7 opcodes (0x27–0x2D) with formally verified laws.

30.1.1 Objects and Morphisms

The partition graph G now carries two maps:

- `pg_modules` : $\text{ModuleID} \rightarrow \text{ModuleState}$ — objects (partition modules)
- `pg_morphisms` : $\text{MorphismID} \rightarrow \text{MorphismState}$ — arrows

A `MorphismState` bundles: source module, target module, coupling data (list of $(\text{nat} \times \text{nat})$ pairs plus a label), and an identity flag.

30.1.2 The Seven Categorical Opcodes

- `MORPH(dst, src, tgt, idx,  $\Delta\mu$ )`: create morphism $\text{src} \rightarrow \text{tgt}$, write morphism ID to `regs[dst]`. Costs $\Delta\mu$.
- `COMPOSE(dst, f, g,  $\Delta\mu$ )`: compose morphisms $f; g$ when $f.\text{target} = g.\text{source}$, write new ID to `regs[dst]`. Costs $\Delta\mu$. Errors if endpoints mismatch.
- `MORPH_ID(dst, m,  $\Delta\mu$ )`: create identity morphism id_m for module m (diagonal relation). Costs $\Delta\mu$.
- `MORPH_DELETE(f,  $\Delta\mu$ )`: remove morphism f . Cascades on module deletion via `graph_cascade_delete_mor`.
- `MORPH_ASSERT(f, prop, cert,  $\Delta\mu$ )`: cert-setter. Costs $S(\Delta\mu) \geq 1$ unconditionally — even for failed attempts (morphism does not exist). This is the No Free Insight law applied to relational claims.
- `MORPH_TENSOR(dst, f, g,  $\Delta\mu$ )`: tensor product $f \otimes g$. Requires that the union modules $\text{src}(f) \cup \text{src}(g)$ and $\text{tgt}(f) \cup \text{tgt}(g)$ already exist in the graph.
- `MORPH_GET(dst, f, sel,  $\Delta\mu$ )`: read field from morphism f : selector 0 = source, 1 = target, 2 = coupling length, 3 = `is_identity`.

30.1.3 Category Laws

Theorem 30.1 (Relational Composition is Associative (kernel/CategoryLaws.v) — (S)). *For any three morphisms $f : A \rightarrow B$, $g : B \rightarrow C$, $h : C \rightarrow D$ with composable endpoints:*

$$(f; g); h = f; (g; h)$$

in the relational sense (coupling pairs of the composite). Zero Admitted.

Theorem 30.2 (Graph Composition Satisfies Laws (kernel/CategoryBridge.v) — (S)). *The operation `graph_compose_morphisms` satisfies associativity and unitality. Additionally:*

$$\text{categorical_extension_nofi_consistent}$$

MORPH and COMPOSE can carry $\Delta\mu = 0$ and are not in the mandatory-cost class. The NoFI policy enters through MORPH_ASSERT: asserting that a built morphism chain exists and activating the cert channel always costs $S(\Delta\mu) \geq 1$. You can build the chain for free; you cannot certify it for free.

Theorem 30.3 (Tensor Bifunctionality (kernel/CategoryMonoidal.v) — (S)). *`graph_tensor_morphisms` is a bifunctor: it satisfies the interchange law*

$$(f \otimes g); (h \otimes k) = (f; h) \otimes (g; k)$$

for composable, disjoint pairs.

30.1.4 The Categorical Separation Theorem

Theorem 30.4 (Categorical Separation (kernel/PartitionSeparation.v, §10) — (S)). *There exist states s_1, s_2 such that:*

$$s_1.\text{vm_regs} = s_2.\text{vm_regs} \wedge s_1.\text{vm_mem} = s_2.\text{vm_mem} \wedge s_1.\text{vm_mu} = s_2.\text{vm_mu} \wedge s_1.\text{vm_err} = s_2.\text{vm_err}$$

but $s_1.\text{vm_graph.pg_morphisms} \neq s_2.\text{vm_graph.pg_morphisms}$.

Proof. Construct s_1 with `pg_morphisms` = $[(0, ms)]$ for any morphism state ms , and s_2 with `pg_morphisms` = $[]$. Arrange identical `vm_regs`, `vm_mem`, `vm_mu`, `vm_err` by choosing costs and instruction sequences accordingly. Distinction by `discriminate`. $\square$ $\square$

Remark 30.1 (Significance). A classical machine (exposing only registers, memory, and a cost counter) cannot distinguish s_1 from s_2 . The Thiele Machine distinguishes them in one probe instruction: MORPH_DELETE 0 succeeds on s_1 and errors on s_2 . The categorical layer records *how* knowledge was derived, not just *what* the final values are.

This is the machine's claim to being a *knowledge auditor*: any party claiming to have verified a structural relation must carry a μ -receipt $\geq$ the minimum construction cost. The receipt is unforgeable by the NoFI theorem applied to MORPH_ASSERT.

30.2 The Conservation as Functor (Archived)

Theorem 30.5 (Conservation as Functor (Universe.v — reference-only)). *Define two categories:*

- $\mathbf{C}_{\text{phys}}$: *Objects* = universe states (particle momenta lists). *Morphisms* = paths of momentum-conserving interactions.
- $\mathbf{C}_{\text{logic}}$: *Objects* = total momentum values. *Morphisms* = equality proofs.

The functor $F : \mathbf{C}_{\text{phys}} \rightarrow \mathbf{C}_{\text{logic}}$ maps $F(s) = \sum s$ (total momentum). Then:

$$\text{Path}(s_1, s_2) \implies \sum s_1 = \sum s_2$$

Conservation of momentum emerges as the functorial image. Observation is a structure-preserving map from physics to logic.

Chapter 31

Audit Infrastructure

Theorem 31.1 (CatNet Integrity (CatNet.v — reference-only)). *Adding a new entry to a valid hash-chain audit log produces a valid chain. If the logic oracle detects inconsistency, execution halts and no further entries are written (paradox halting).*

Chapter 32

Falsifiable Predictions

32.1 Linear Scaling of Structural Cost

Axiom 32.1 (Linear Scaling (kernel/FalsifiablePrediction.v)). The μ -cost of maintaining coherence of N entangled qubits scales as $O(N)$ per time step.

Prediction: Large-scale quantum computers will encounter a fundamental (not merely technical) decoherence noise floor proportional to entanglement complexity.

32.2 CHSH Regime Separation

Prediction: Experiments probing the boundary between quantum ($|S| \leq 2\sqrt{2}$) and supra-quantum ($|S| > 2\sqrt{2}$) correlations should find that exceeding the Tsirelson bound requires measurably higher energy dissipation, scaling as:

$$\Delta E \geq k_B T \ln 2 \cdot (|S| - 2\sqrt{2})$$

32.3 Metric Deformation

Since d_μ depends on information content, regions of high structural complexity effectively expand the metric.

Prediction: In high-complexity computations, effective signal latency will increase relative to vacuum propagation. (This prediction relies on interpreting d_μ as physically meaningful, which is currently an analogy rather than a derived result.)

32.4 Architectural Permanence

Theorem 32.1 (Optimal Quartet (ArchTheorem.v — reference-only)). *The four partition discovery strategies (Louvain, Spectral, Degree, Balanced) achieve classification accuracy > 90% across all problem classes. No alternative configuration exceeds this quartet's accuracy. The configuration is architecturally final.*

32.5 Coupling Constant Prediction

Definition 32.1 (Thiele α Limit (thiele_manifold/PhysicalConstants.v)). The asymptotic density of self-referential programs in the n -bit state space:

$$\alpha = \lim_{n \rightarrow \infty} \frac{A_{\text{interaction}}(n)}{V_{\text{spacetime}}(n)} = \lim_{n \rightarrow \infty} \frac{n+1}{2^n}$$

This converges to 0, but the *non-asymptotic* structure at physical scales should relate to coupling constants.

Appendix A

Proof File Index

The table below maps each theorem to its Coq source file. Entries marked — *reference-only* are theoretical results that belong to an extended corpus; the corresponding Coq files are not present in the active kernel tree and their proofs are not machine-checked in the current build. Everything unmarked lives in the active corpus and compiles clean.

Result	Coq Source
No Free Insight (Thm. 5.1)	nofi/NoFreeInsight_Theorem.v
μ -Chaitin	nofi/MuChaitinTheory_Theorem.v
Cost = Complexity	theory/CostIsComplexity.v
No Free Lunch	theory/NoFreeLunch.v
No Arbitrage	kernel/NoArbitrage.v
TM Embedding	thielemachine/coqproofs/Embedding_TM.v
TM $\rightarrow$ Minsky	modular_proofs/TM_to_Minsky.v
Strict Subsumption	thielemachine/coqproofs/Subsumption.v
Semantic Strictness	thielemachine/coqproofs/ThieleFoundations.v
Confluence	thielemachine/coqproofs/Confluence.v
Tensoriality	thielemachine/coqproofs/ThieleUnificationTensor.v
μ -Conservation	kernel/MuLedgerConservation.v
Receipt Integrity	kernel/ReceiptIntegrity.v
Noether / Gauge	kernel/KernelNoether.v
Gauge Trace Preservation	thielemachine/coqproofs/RepresentationTheorem.v
2D Necessity	quantum_derivation/TwoDimensionalNecessity.v
Complex Necessity	quantum_derivation/ComplexNecessity.v
Born Rule	kernel/BornRule.v
Born Rule (Tensor)	kernel/BornRuleFromSymmetry.v
Unitarity	kernel/Unitarity.v
Purification	kernel/Purification.v
No-Cloning	kernel/NoCloning.v
No-Cloning (Machine-Native)	kernel/NoCloningFromMuMonotonicity.v
Observation Irreversibility	quantum_derivation/ObservationIrreversibility.v
Collapse Determination	quantum_derivation/CollapseDetermination.v
Tsirelson Derivation	kernel/TsirelsonDerivation.v
Tsirelson (Algebraic)	kernel/TsirelsonFromAlgebra.v
Tsirelson (General)	kernel/TsirelsonGeneral.v
CHSH Separation	thielemachine/verification/Deliverable_CHSHSeparati
Emergent Schrödinger	physics_exploration/EmergentSchrodinger.v
Planck Consistency	physics_exploration/PlanckDerivation.v
Spacetime Emergence	kernel/SpacetimeEmergence.v
μ -Geometry	kernel/MuGeometry.v
4D Simplicial Complex	kernel/FourDSimplicialComplex.v
Metric from μ -Costs	kernel/MetricFromMuCosts.v
Riemann Curvature Tensor	kernel/RiemannTensor4D.v
Christoffel Symbols	kernel/RiemannTensor4D.v
Einstein Tensor	kernel/RiemannTensor4D.v
Stress-Energy Tensor	kernel/EinsteinEquations4D.v
Curvature from μ -Gradients	kernel/EinsteinEquations4D.v
Stress-Energy Conservation	kernel/EinsteinEquations4D.v
Einstein Equations (Vacuum)	kernel/EinsteinEquations4D.v
Einstein Equations (General)	kernel/EinsteinEquationsFull.v
Discrete Gaussian Curvature	kernel/MetricFromMuCosts.v
Information Causality	kernel/InformationCausality.v
Landauer	thermodynamic/LandauerDerived.v
Dissipative Embedding	thielemachine/coqproofs/DissipativeEmbedding.v
Physics Embedding	thielemachine/coqproofs/PhysicsEmbedding.v
Wave Embedding	thielemachine/coqproofs/WaveEmbedding.v
Self-Reference	self_reference/SelfReference.v
Thiele Manifold	thiele_manifold/ThieleManifold.v
Genesis	theory/Genesis.v
Full Isomorphism	kernel/ThreeLayerIsomorphism.v
Curry–Howard–Thiele	thielemachine/coqproofs/LogicIsomorphism.v
Logic Is Physics	theory/LogicToPhysics.v
Functor Soundness	isomorphism/coqproofs/Universe.v
CatNet Integrity	catnet/coqproofs/CatNet.v
Amortization	thielemachine/coqproofs/AmortizedAnalysis.v
Optimal Quartet	theory/ArchTheorem.v
Falsifiable Prediction	kernel/FalsifiablePrediction.v
Physical Constants	thiele_manifold/PhysicalConstants.v
Halting Undecidability	kernel/OracleImpossibility.v
Oracle Cost Bound	kernel/OracleImpossibility.v

Appendix B

Epistemological Classification

The following table classifies every major result by its epistemological status.

Result	Status	Key Assumption
<i>Unconditional structural theorems about the machine</i>		
No Free Insight	(S)	Module type contract
μ -Chaitin	(S)	Module type contract
Cost = Complexity	(S)	Prefix-free coding
No Free Lunch	(S)	Faithful representation
No Arbitrage $\Rightarrow$ Potential	(S)	Additivity + no-arbitrage (toy model)
TM Embedding	(S)	Standard simulation
Strict Subsumption	(S)	Partition structure
Halting Undecidability	(S)	Diagonalization
Oracle Cost	(S)	Definitional (soundness $:= \text{cost} \geq 1$)
Confluence	(S)	Module independence
μ -Conservation	(S)	Machine semantics only
μ -Ledger Necessity	(S)	Machine semantics + explicit witnesses
μ -Ledger Minimality	(S)	Named projection lattice in <code>NecessityAbstract.v</code>
Receipt Integrity	(S)	Hash-chain model
Gauge / Noether	(S)	Definitional (record structure)
Genesis	(S)	Definitional isomorphism
Three-Layer Isomorphism / comparison contract	(C)	Backend decoders satisfy the shared observable projection
Curry–Howard–Thiele	(S)	Interpretation map
Curvature from μ -Gradients	(S)	Lemma 24.5
Vacuum Implies Zero Stress-Energy	(S)	Lemma 24.6
Einstein Equations (Vacuum)	(S)	Theorem 24.7
Einstein Equations (General full tensor)	(C)	Named off-diagonal Ricci premise in <code>EinsteinEquations</code>
Affine Off-Diagonal Ricci Zero (vertex 1)	(S)	Non-uniform isotropic boundary 4-simplex; affine operator
Full Tensor EFE at Vertex 1	(S)	<code>module_structural_mass s 1 = 1</code>
Outer Vertex Characterization	(S)	Same affine operator, outer vertices $\{0, 2, 3, 4\}$
Discrete Differential Geometry	(S)	Definitions only
Partition Ops Free / Certification Non-Free	(S)	Machine semantics only
CHSH Trial Is Tier 0	(S)	Machine semantics only
Three-Tier Observation Taxonomy	(S)	Machine semantics only
Locally Consistent $\Rightarrow$ Separable Coupling	(S)	Coupling definition
Local Coupling Bound ($ S \leq 2$)	(S)	Coupling definition
$S > 2$ Rules Out Local Couplings	(S)	Coupling definition
RTL Gap Count = 0	(S)	Machine semantics + hardware spec
Classical $P \subseteq \mu P$	(S)	μ -complexity definitions
<i>Conditional derivations (valid given stated axioms)</i>		
2D Necessity	(C)	Superposition axiom (Axiom 12.1)
Complex Necessity	(C)	2D + norm preservation
Born Rule Uniqueness	(C)	Tensor product consistency (<code>BornRuleFromSymmetry.v</code>)
Unitarity from $\mu = 0$	(C)	Info conservation (one direction only)
No-Cloning	(C)	Purity = information
Purification	(C)	Bloch sphere model
Observation Irreversibility	(C)	REVEAL semantics
Collapse Determination	(C)	Entropy = log dim
Emergent Schrödinger	(C)	2D + antisymmetry
Landauer	(C)	Second law <i>assumed</i> as record field
Dissipative Embedding	(C)	Irreversibility model
Self-Reference	(C)	Dimension = complexity
CHSH Separation	(C)	Numerical witnesses
Tsirelson Bound	(C)	Algebraic derivation (<code>TsirelsonFromAlgebra.v</code>)
<i>Consistency relations (definitions verified, not predictions)</i>		
Planck's Constant	(R)	$\tau_\mu := h/(4E_L)$
Information Causality	(R)	Circular: equivalence assumed as hypothesis
μ -Geometry	(R)	Pseudometric (absolute value on $\mathbb{Z}$)
Spacetime Locality	(R)	Data isolation in partition graph
Born Rule (<code>BornRule.v</code>)	(R)	μ -consistent hypothesis unused

Legend: **(S)** = unconditional theorem about the machine; **(C)** = valid derivation conditional on stated axiom; **(R)** = internal consistency check.